

ASSIGNMENT – 1

System Design — Movie Ticket Booking System

Course: B.Tech. CSE | Semester: 5

Subject: System Design | Subject Code: TCS-504

Submission Date: 07-September-2026

Submitted By: _____

Roll No.: _____

Section: _____

Submitted To: _____

A. Requirement Analysis

Functional Requirements

FR1 — Movie Listing: The system shall display all movies currently playing along with their title, language and duration.

FR2 — Show Listing: For a selected movie, the system shall display its available shows with screen number and start time.

FR3 — Seat Display: For a selected show, the system shall display the seat layout and indicate each seat as AVAILABLE or BOOKED.

FR4 — Booking: A customer can select one or more seats. If any selected seat is already BOOKED, the booking is rejected and no seat state changes. Booking is confirmed only after successful payment.

FR5 — Pricing: The system shall calculate the total amount using seat type: SILVER ■150, GOLD ■250 and PLATINUM ■400.

FR6 — Payment: Exactly one payment method (UPI, Card or Cash) is selected per booking. A failed payment must not confirm the booking and the selected seats remain available.

FR7 — Ticket: After successful payment, the system shall print a ticket containing booking ID, movie, screen, time, seat numbers and total amount.

FR8 — Cancellation: A customer can cancel a confirmed booking, after which all seats in that booking become AVAILABLE again.

Non-Functional Requirements

Modularity: each major class is kept in a separate source file with a focused responsibility.

Extensibility: new payment methods can be introduced through Payment without editing existing payment implementations.

Reliability: failed payments do not confirm bookings and cancelled bookings release seats.

Input Validation: invalid choices and seat numbers are handled clearly without crashing.

B. Noun–Verb Analysis

Noun found	Class?	Reason
Movie	Yes	Own data and identity: title, language and duration.
Seat	Yes	Own data and identity: seat number and seat type.
Cinema	Yes	Represents the theatre and owns its screens.
Screen	Yes	Represents one auditorium and owns its physical seats.
Show	Yes	Represents one screening of a movie on a screen at a time.
ShowSeat	Yes	Stores the status of one seat for one show.
Customer	Yes	Represents the customer making a booking.
Booking	Yes	Stores booking identity, show, seats, amount and status.
Payment	Yes	Abstract contract for payment processing.
UpiPayment	Yes	Concrete payment implementation.
CardPayment	Yes	Concrete payment implementation.
CashPayment	Yes	Concrete payment implementation.
PriceCalculator	Yes	Encapsulates price calculation behaviour.
TicketPrinter	Yes	Responsible only for ticket output.
BookingService	Yes	Coordinates the booking workflow.
Seat layout	No	It is a view of a Show's ShowSeat objects, not an independent domain object.

Verb → responsible class: list movies → BookingService; list shows → BookingService; display seats → Show; book/release → BookingService/ShowSeat; calculate price → PriceCalculator; pay → Payment subclasses; print ticket → TicketPrinter; cancel booking → BookingService.

C. Classes and Responsibilities

Movie — Knows: title, language, duration. Does: provides movie information. Must NOT: handle booking or payment.

Seat — Knows: seat number and tier. Does: provides physical seat information. Must NOT: manage booking state.

Screen — Knows: screen number, name and seats. Does: owns and manages physical seats. Must NOT: process payment.

Cinema — Knows: cinema name and screens. Does: owns screens and represents the theatre. Must NOT: calculate booking price.

Show — Knows: movie, screen, show ID, start time and ShowSeats. Does: represents one screening and displays its seats. Must NOT: process payment.

ShowSeat — Knows: seat number, tier and booked status. Does: book and release a seat for one show. Must NOT: calculate the whole booking amount.

Customer — Knows: name and phone. Does: represents the customer. Must NOT: directly modify seat status.

Booking — Knows: booking ID, show, customer, selected seats, amount and status. Does: stores booking state. Must NOT: print the ticket.

Payment (abstract) — Knows: payment contract. Does: defines pay() and payment method. Must NOT: control seat state.

UpiPayment / CardPayment / CashPayment — Knows: method-specific payment details. Does: implement payment. Must NOT: manage booking workflow.

PriceCalculator — Knows: seat pricing constants. Does: calculates totals. Must NOT: book seats.

TicketPrinter — Knows: booking information. Does: formats and prints a ticket. Must NOT: change booking state.

BookingService — Knows: cinema, shows and bookings. Does: orchestrates listing, booking and cancellation. Must NOT: contain the internal logic of each payment method.

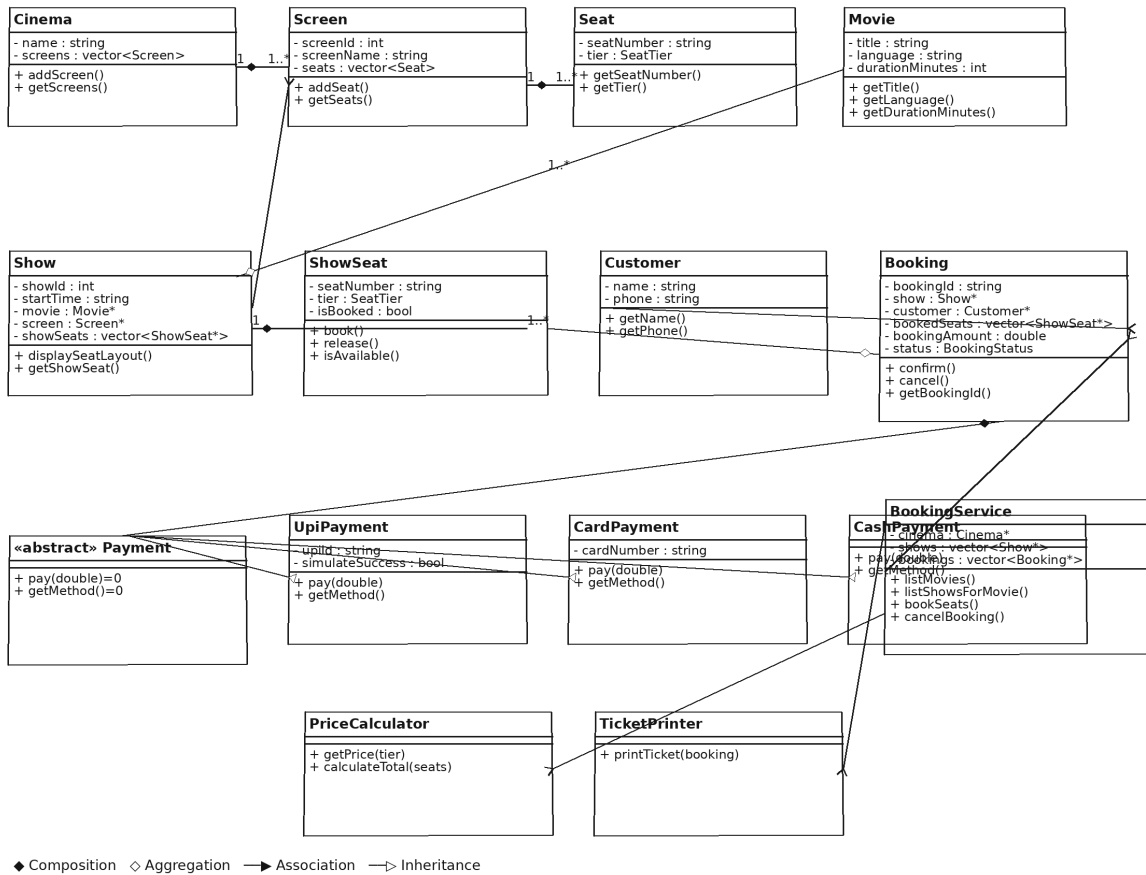
D. Relationship Table

Pair	Relationship	Lifetime-test justification
Cinema — Screen	Composition	Cinema owns its Screen objects. When the Cinema object is destroyed, its owned Screen objects are destroyed.
Screen — Seat	Composition	Screen stores and owns its physical Seat objects. The seats belong to that auditorium in this model.
Show — Movie	Aggregation	A Show refers to an existing Movie. Destroying a Show does not destroy the Movie, which can be used by other Shows.
Show — Screen	Association	A Show uses a Screen for a screening, but the Screen has an independent lifetime and can host other Shows.
Show — ShowSeat	Composition	Show creates and owns the ShowSeat objects used to track seat status for that screening. They are destroyed with the Show.
Booking — Customer	Association	Customer exists independently and can make multiple bookings. Destroying a Booking does not destroy the Customer.
Booking — ShowSeat	Aggregation	Booking stores references to existing ShowSeat objects. Destroying a Booking does not destroy the ShowSeat objects.
Booking — Payment	Association	The booking uses a Payment object to complete the transaction; the payment implementation can exist independently.
Payment — UpiPayment	Inheritance	UpiPayment is a specialized Payment implementation and can substitute the Payment base type.
BookingService — Booking	Association	BookingService creates and manages workflow around Booking objects but the service is not the lifetime owner.

E. Class Diagram

Monochrome hand-drawn style diagram with the required classes, attributes, methods, visibility markers, multiplicities and relationship notation.

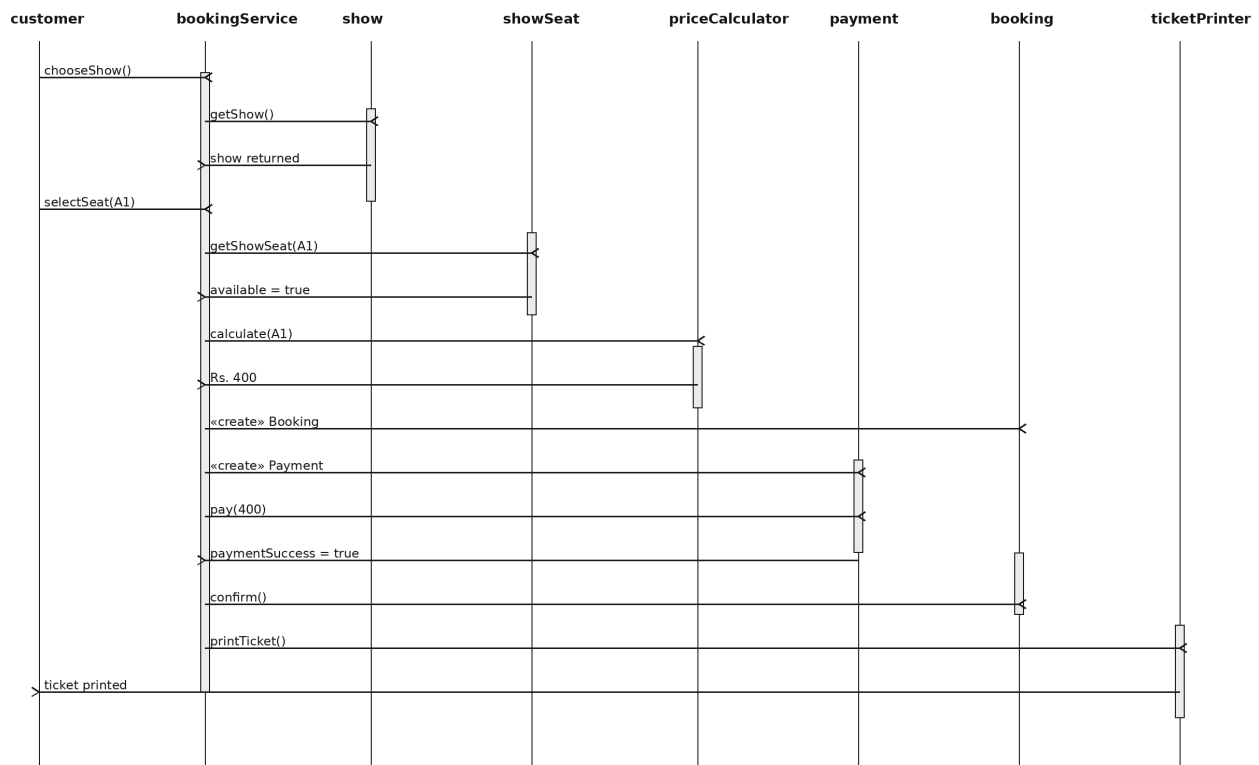
Class Diagram — Movie Ticket Booking System



F. Sequence Diagram

Use case: customer books one seat and pays by UPI.

Sequence Diagram — Customer Books 1 Seat and Pays by UPI



Required flow: choose show → get show → select seat → availability check → price calculation → create Booking → create Payment → pay → confirm → print ticket.

F. Modular Working Code + Demo Run

The accompanying **bookmyshow_project** folder contains the working C++ source and executable. The source is organised as one class per .cpp file and uses no header files.

Source files: _Booking.cpp, _BookingService.cpp, _CardPayment.cpp, _CashPayment.cpp, _Cinema.cpp, _Customer.cpp, _Movie.cpp, _Payment.cpp, _PriceCalculator.cpp, _Screen.cpp, _Seat.cpp, _Show.cpp, _ShowSeat.cpp, _TicketPrinter.cpp, _UPIPayment.cpp, main.cpp

Build: g++ main.cpp -std=c++17 -Wall -Wextra -o main.exe

OOP concepts: encapsulation, abstraction, inheritance, runtime polymorphism, compile-time polymorphism, static member, this keyword, composition, aggregation and association.

Edge cases: already-booked seat is rejected; failed payment leaves seats available; cancellation releases seats; invalid seat/show/menu input gives a clear message.

```
PS C:\Users\yashd\OneDrive\Desktop\BookMyShow_Project> g++ main.cpp -o main
PS C:\Users\yashd\OneDrive\Desktop\BookMyShow_Project> ./main

=====
CINEMA TICKETING SYSTEM DEMO RUN
=====

=== Seat Layout for Oppenheimer (07:00 PM) ===
[A1 : AVAILABLE] [A2 : AVAILABLE] [B1 : AVAILABLE]
=====

>>> [DEMO USE CASE] Booking seat A1 via UPI...
[UPI Gateway] Authorized Rs. 400 from VPA: yash@okaxis

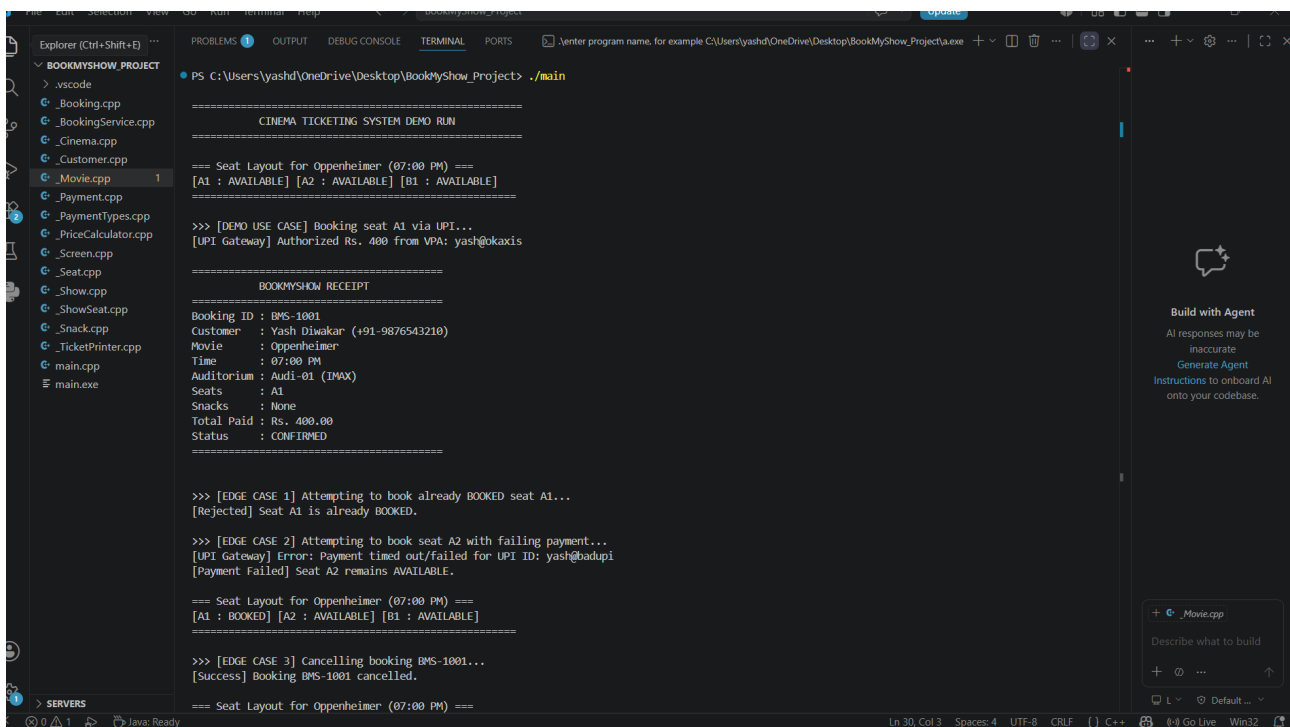
=====
BOOKMYSHOW RECEIPT
=====
Booking ID : BMS-1001
Customer  : Yash Diwakar (+91-9876543210)
Movie     : Oppenheimer
Time      : 07:00 PM
Auditorium : Audi-01 (IMAX)
Seats     : A1
Snacks    : None
Total Paid : Rs. 400.00
Status    : CONFIRMED
=====

>>> [EDGE CASE 1] Attempting to book already BOOKED seat A1...
[Rejected] Seat A1 is already BOOKED.

>>> [EDGE CASE 2] Attempting to book seat A2 with failing payment...
[UPI Gateway] Error: Payment timed out/failed for UPI ID: yash@badupi
[Payment Failed] Seat A2 remains AVAILABLE.

=== Seat Layout for Oppenheimer (07:00 PM) ===
[A1 : BOOKED] [A2 : AVAILABLE] [B1 : AVAILABLE]
=====

>>> [EDGE CASE 3] Cancelling booking BMS-1001...
[Success] Booking BMS-1001 cancelled.
```



```
>>> [EDGE CASE 1] Attempting to book already BOOKED seat A1...
[Rejected] Seat A1 is already BOOKED.

>>> [EDGE CASE 2] Attempting to book seat A2 with failing payment...
[UPI Gateway] Error: Payment timed out/failed for UPI ID: yash@badupi
[Payment Failed] Seat A2 remains AVAILABLE.

=== Seat Layout for Oppenheimer (07:00 PM) ===
[A1 : BOOKED] [A2 : AVAILABLE] [B1 : AVAILABLE]
=====

>>> [EDGE CASE 3] Cancelling booking BMS-1001...
[Success] Booking BMS-1001 cancelled.

=== Seat Layout for Oppenheimer (07:00 PM) ===
[A1 : AVAILABLE] [A2 : AVAILABLE] [B1 : AVAILABLE]
=====

>>> [EDGE CASE 4a] Selecting non-existent seat Z99...
[Error] Seat "Z99" does not exist!

>>> [EDGE CASE 4b] Selecting invalid show index 5...
[Error] Invalid show selection!

Demo completed safely with 0 crashes.
PS C:\Users\yashd\OneDrive\Desktop\BookMyShow_Project> █
```

G. SOLID Mapping

S — Single Responsibility: Movie stores movie data, PriceCalculator calculates prices, TicketPrinter prints tickets and BookingService coordinates the workflow.

O — Open/Closed: A new payment type can be introduced as another Payment subclass without changing existing UPI/Card/Cash implementations.

L — Liskov Substitution: UpiPayment, CardPayment and CashPayment can be used wherever the Payment abstraction is expected.

I — Interface Segregation: The Payment abstraction exposes only the operations common to payment methods; unrelated operations are not forced on subclasses.

D — Dependency Inversion: BookingService receives a Payment abstraction rather than depending on a concrete payment implementation.

Deliberately not done: A database layer was not added because this is a small single-cinema console application. Data is kept in memory so the design remains focused on OOP, relationships and SOLID.

Clean code: meaningful names, focused functions, named price constants, centralized validation, no direct public seat-status modification and clear input checks.